Array 27 (2025) 100458

Contents lists available at ScienceDirect

Array

journal homepage: www.elsevier.com/locate/array

A reinforcement learning based fuzzing technique for binary programs vulnerabilities detection

Guoyan Cao [a,*], Yanhui Ma [b], Mengjiao Geng [a]

[a] School of Cybersecurity in Northwestern Polytechnical University, Shaanxi, China
[b] China Aviation Industry Corporation Jinan Special Structure Research Institute, Shandong, China

ARTICLE INFO

Keywords:
Binary program
Fuzzing
Reinforcement learning
Vulnerability probability

ABSTRACT

Binary programs are susceptible to vulnerabilities that can lead to unauthorized access, data breaches, and system damage. Fuzzing is a promising technique for identifying vulnerabilities in binary programs. However, fuzzing is time-consuming and inefficient as many seeds are random and recurrently executed. This paper proposes a novel approach called Vulnerable State Guided Fuzzing (VSGFuzz) employs a heuristic mechanism for generating seeds to optimize vulnerability detection. This mechanism assesses the vulnerable probability of each function within the target binary program and employs reinforcement learning to mutate seeds based on a comprehensive reward calculation algorithm considering vulnerable probability and coverage assessment. Experiments evaluating VSGFuzz compared with other typical methods on several datasets demonstrated the superiority of the proposed method over other methods.

1. Introduction

Binary programs have become the cornerstone of modern computing by providing efficient computing power and possessing cross-platform characteristics. Security considerations regarding binary programs are crucial for the reliability of modern computing platforms, guarding against unauthorized access, leakage of sensitive information, and system damage. However, binary programs may suffer from numerous unknown vulnerabilities that can compromise the security and reliability of systems. These vulnerabilities may arise from insecure usage of binary programs or dangerous operations involving them, such as data fabrications, service interruptions, and system attacks. Binary vulnerability playing as a form of malwares poses a significant security treat to computer environments and network surroundings [1], shown as in Fig. 1.

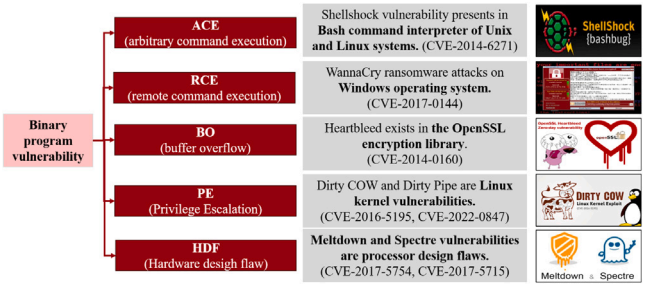


Fig. 1. Examples of binary program vulnerabilities.

Fuzzing is one of the influential and powerful techniques for detecting and discovering vulnerabilities in binary programs [2]. Fuzzing involves executing unexpected inputs, carefully observing program feedback and crash statuses, capturing coverage paths associated with the inputs, and repeatedly testing mutated inputs to uncover vulnerabilities in the target program. While this process is effective, it is also time-consuming and inefficient. Recently, significant research efforts have been focused on enhancing the efficiency of fuzzing by refining coverage-based methodologies under the assumption that greater path coverage increases the likelihood of discovering vulnerabilities. However, research methodologies that solely rely on path coverage often fail to produce optimal outcomes of high quality [3]. This is because a significant number of covered paths, resulting from code logic, conditional constraints, or error handling conditions, do not contribute to finding potential vulnerabilities. Furthermore, the proportion of hidden vulnerabilities within the program code itself is relatively small. For example, an investigation by Shin and Williams [4] revealed that vulnerabilities were present in less than 5% of the source code files in Mozilla Firefox.

In light of the above considerations, this paper proposes a vulnerable state-guided fuzzing methodology (VSGFuzz) to enhance the

* Corresponding author.
E-mail addresses: guoyan.cao@nwpu.edu.cn (G. Cao), vivian2855@163.com (Y. Ma), gmj_2000@163.com (M. Geng).

https://doi.org/10.1016/j.array.2025.100458
Received 3 November 2024; Received in revised form 7 July 2025; Accepted 8 July 2025
Available online 21 July 2025

efficiency of vulnerability detection. VSGFuzz is modeled as a Markov decision process, employing a reinforcement learning algorithm to direct superior mutation operations for generating higher-quality seed inputs. This approach optimizes the fuzzing process, making it more effective in identifying vulnerabilities in binary programs. VSGFuzz also leverages the RL algorithm to guide the mutation of input seeds with a higher likelihood of reaching vulnerable code segments. Unlike the equal seed mutation strategy commonly used by existing coverage-based fuzzers, VSGFuzz emphasizes triggering crashes or attaining higher seed values. This approach capitalizes on the strengths of vulnerability prediction and coverage while mitigating their respective limitations. The contributions of this paper are summarized as follows:

(1) A novel fuzzing method is proposed to enhance coverage efficiency and reduce runtime overhead. The fuzzing process is modeled as a Markov decision-making process, with a Reinforcement Learning mechanism designed to guide seed mutation, thus improving fuzzing efficiency and reducing overhead.

(2) Design a new metric for assessing the quality of seed files that not only evaluates vulnerability probability but also considers path weight and HugeScore to characterize the probability of program crashes.

(3) VSGFuzz has been demonstrated to detect more vulnerabilities in both general datasets and real-world programs compared to other methods. We implemented VSGFuzz on the LAVA-M dataset as well as several real-world programs to evaluate its fuzzing performance on binary programs. The results illustrated that VSGFuzz identified more vulnerabilities than state-of-the-art methods.

2. Related work

Fuzzing techniques have become integral for software testing and vulnerability discovery [5]. In this section, we have compiled some research in fuzzing to build upon existing work.

2.1. Fuzzing

Fuzzing techniques originated in the 1980s when Barton Miller and his colleagues first introduced the concept of fuzz testing, applying it to UNIX systems [6]. In the early 2000s, the mutation-based fuzzing approach emerged, involving a random or semi-random mutation of input data to generate test cases [7]. In 2005, the book “Fuzzing for Software Security Testing and Quality Assurance” by Takanen, DeMott, and Miller provided a comprehensive overview of fuzzing techniques in software security testing [8]. The year 2006 saw the introduction of the SAGE (Scalable Automated Guided Execution) fuzzing framework, enabling automated fuzz testing in large-scale programs [9]. In 2010, TaintScope [10] employed dynamic taint analysis and symbolic execution to test x86 binary programs. The next year, SimFuzz [11] utilized test case similarity metrics to explore program semantics deeply.

A significant advancement occurred in 2013 with the release of American Fuzzy Lop (AFL) [12], which employed coverage-based strategies and intelligent input mutation to discover program vulnerabilities. Dowser [13], a guided fuzzier, integrates taint tracking, program analysis, and symbolic execution to detect buffer overflow and underflow vulnerabilities. By 2016, the integration of symbolic execution and fuzzing gained popularity, enhancing the accuracy and efficiency of fuzz testing [14]. QuickFuzz [15] leveraged existing Haskell libraries to test various file formats without manual specifications in 2017 efficiently. Another fuzzier, VUzzer [16], on the other hand, utilizes control-flow and data-flow analysis to pinpoint bugs in deeper program paths, assigning increased significance to these intricate code components. Another strategy entails employing program analysis to support fuzzing efforts [17]. V-Fuzz [18] is designed to identify a wider array of vulnerabilities than Dowser.

2.2. Learning-based fuzzing

In recent years, researchers have combined machine learning with traditional fuzzing techniques, integrating machine learning methods into the stages of seed file generation and mutation strategy selection in the fuzzing process. Those integrations with different machine learning approaches, such as supervised (Regress and Classification), unsupervised (Clustering and Desity estimation), semi-supervised and reinforcement learning, bring in varies features, perspecitves and evolutions on fuzzing techniques [19].

In the seed file generation process, SmartSeed [20] utilizes Generative Adversarial Networks (GANs) to generate new binary samples from valid sample data identified by AFL. This approach triggers more crashes and execution paths than AFL and other selection strategies. REINAM [21] employs a reinforcement learning algorithm to generate program input grammar, thereby improving the quality of generated variation samples. In addition to traditional seed input in file format, software testing engineering encompasses other forms of seed input. For instance, Luong et al. [22] employed the Q-Learning algorithm to test Android applications. This involved selecting and arranging operations to trigger GUI event sequences, resulting in higher code coverage and more application errors being triggered. Deng et al. [23] use LLMs for black-box fuzzing testing Mutation, proving the effectiveness of pre-trained LLMs not only for seed generation but also for mutation. Cheng et al. [24] introduces a generative model built on a recurrent neural network (RNN) framework to provide high-quality sequences for PDF-based program targets. She et al. [25] proposed using a gradient-guided input generation scheme and procedural smoothing techniques, in which the neural network learns smooth approximations. Scott et al. [26] leverages multi-agent RL guidance to improve SMT solver performance. The core idea revolves around training a single or a group of agent explorers to maximize reward signals that indicate the effectiveness of the generated inputs, particularly in achieving high code coverage or identifying bugs.

In selecting mutation strategies, researchers have been exploring integrating reinforcement learning techniques to enhance fuzz-ing efficiency. Bottinger [27] and colleagues utilized the Deep Q Network (DQN) of reinforcement learning to fuzz the Japanese standard program pdftotext. This approach employs strings as the state and six self-defined mutation methods as actions. Compared to the traditional random mutation action selection strategy, this design significantly improves the code coverage rate and reduces the running time.FUZZBOOST [28] shares a similar concept. FuzzerGym [29] integrates libFuzzer [30] and Double Deep Q Network (DDQN) via the Remote Procedure Call (RPC) method for target programs libjpeg, libpng, boringssl, re2, and sqllite. In fuzz testing, libFuzzer is guided to select the mutation function through the reinforcement learning algorithm. This approach leads to a significant improvement in code coverage compared to the original random selection strategy. Kuznetsov et al. [31] also employed the DQN algorithm to select mutation actions, resulting in a reduction of approximately 30% in the number of sample mutations required to detect vulnerabilities. Zhang et al. [32] introduced two key enhancements to traditional grey-box fuzzing: a multi-player, multi-armed bandit model and a non-dominated sorting genetic algorithm for fuzzing. Wang et al. [33] use Deep Neural Network (DNN) to optimize the selection strategy of samples and improve the coverage of vulnerable paths. Suzzer [34] predicts the vulnerable probability of functions based on a deep learning-based model and gives each basic block in the vulnerable function a static score. Then, for each input, the sum of the static score of all the basic blocks on its execution path is calculated, and the inputs are prioritized with higher scores.

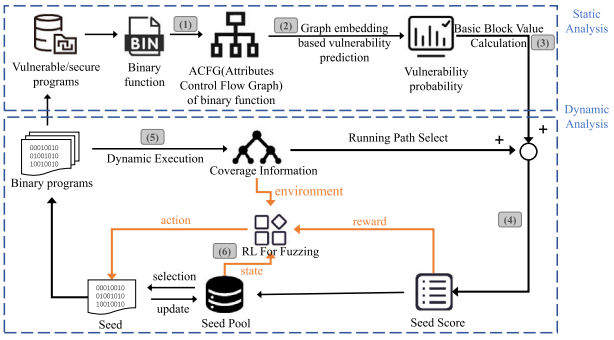


Fig. 2. Workflow of the VSGFuzz.

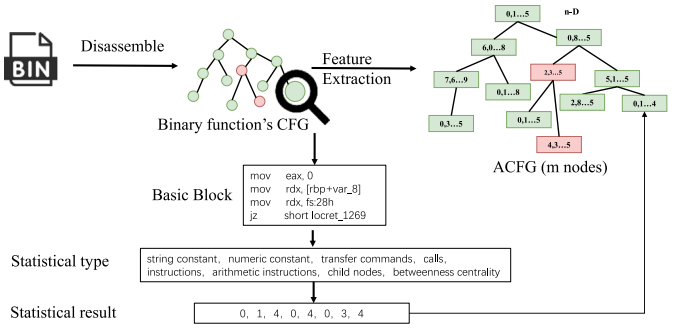


Fig. 3. Procedure for binary programs to extract the ACFG.

3. Overview

This section details the proposed vulnerable state-guided fuzzing methodology (VSGFuzz). As depicted in Fig. 2, VSGFuzz architecture consists of six steps and is grouped into two main processes: (1) static analysis and dynamic analysis.

In the static analysis process, the first step is to formulate binary functions into an ACFG data structure, which can used in a graph embedding network [35,36] to estimate the vulnerability probability of functions within binary programs in the second step. Based on function-level vulnerability probability analysis, a basic block value calculation mechanism is designed to seed evaluation during the fuzzing phase.

An RL-based and vulnerable state-guided fuzzy logic is proposed in the dynamic analysis process. The core part of the fuzzer is finding effective seeds to point to highly vulnerable areas and cover function paths as many as possible. The details of the techniques are introduced in Step 4 - Step 6. Using the RL algorithm, the fuzzing process is trained to select more effective mutations progressively.

4. Methodology of VSGFuzz

4.1. Step 1: Data preprocessing

Vulnerability prediction at the program level cannot capture the nuanced relationship between the program and vulnerable code segments. This limitation hampers the effective mining of vulnerabilities and impedes subsequent mutation processes of the fuzzer. To address this, we adopt function-level analysis to assess program vulnerabilities, and therefore, we need to characterize the specific features of binary functions.

Conventional approaches to binary programs are grouped into three methods: (1) converting function contents into byte arrays, (2) collecting information on the type and quantity of identifiers within a function, and (3) utilizing program abstract syntax trees, among others. However, the former two methods disregard the structural information of the function, while the latter sacrifices its statistical information. To comprehensively account for both the structure and semantic information of functions, based on the third method, we build the Attributes Control Flow Graph (ACFG) [35,37–39] representation to characterize the binary function.

ACFG [35] is a directed graph denoted as

$$g = \langle V, E, \phi \rangle \tag{1}$$

where V represents a set of nodes, E represents a set of edges, and ϕ is a mapping function $\phi : V \rightarrow \Sigma$ that assigns attribute sets Σ to the nodes in the graph. The construction of ACFG relies on a control flow graph (CFG), which employs diagrams to illustrate all possible execution paths within a computer program.

The process of obtaining the ACFG of a binary function is illustrated in Fig. 3. Firstly, the binary program is disassembled to derive the CFG of each function within the program. Subsequently, attribute information is extracted from each basic block, and each basic block is transformed into a numerical vector. By following these steps, the ACFG representation of the function is obtained. These attributes encompass essential components such as string and numerical constants, transfer and call instructions, arithmetic instructions, and child nodes, all of which provide insights into the function's behavior. Furthermore, the betweenness centrality measure captures the significance of a node within the overall graph structure, aiding in the identification of potential vulnerabilities within the function. Both the function's statistical and structural information are stored as vectors in the nodes, serving as attribute vectors for the basic block nodes. Each node in the ACFG possesses an attribute vector with eight dimensions, representing the function as a collection of eight-dimensional vectors. Subsequently, the extracted ACFG of the binary function is utilized as input for the model.

4.2. Step2: Vulnerability probability prediction

In this step, the Neural Network model-based vulnerability probability prediction model is designed to determine the degree of vulnerabilities in binary functions. It produces a vulnerability prediction value called Vulnerable Probability (VP), denoted as $VP \in [0, 1]$, representing the probability of fragility in binary functions. Let M denote the vulnerability prediction model. Each function f_i in the binary program is input to the model M. Assuming the binary program contains a set of n functions, denoted as

$$F = f_1, f_2, \ldots, f_n \tag{2}$$

where $f_i \in F$. Based on the description above, the model's processing formula is expressed as

$$VP_{f_i} = M(f_i) \tag{3}$$

The machine learning-based model is implemented as in Algorithm 1. Given a binary program function, we represent it as an ACFG denoted by $g = \langle V, E, \phi \rangle$. The g representation serves as the input to our model. Each ACFG g consists of an attribute matrix M constructed using selected attributes. The number of attributes per basic block is represented as n. Therefore, M is an $m \times n$ dimensional matrix, where m represents the total number of basic blocks in the graph g.

For an input g, the graph embedding network computes an embedding vector μ_g, which incorporates topological information about the graph. The dimensionality of the embedding vector μ_g is d. N is the adjacent matrix to g. W_1, W_2, W_3 are neural network weights. Then, the embedding vector μ_g is calculated by

$$\mu_g = F(M \times W_1, \sigma_j(N \times \mu_j)) \tag{4}$$

where F is defined as $tanh$ or $sigmoid$. This formulation describes computing the embedding vector $\mu^{(i)}$ for each iteration $t \in T$. The initial embedding vector $\mu^{(0)}$ is set to zero. After T iterations, we get the graph embedding vector $\mu^{(T)}$. And the final graph embedding vector

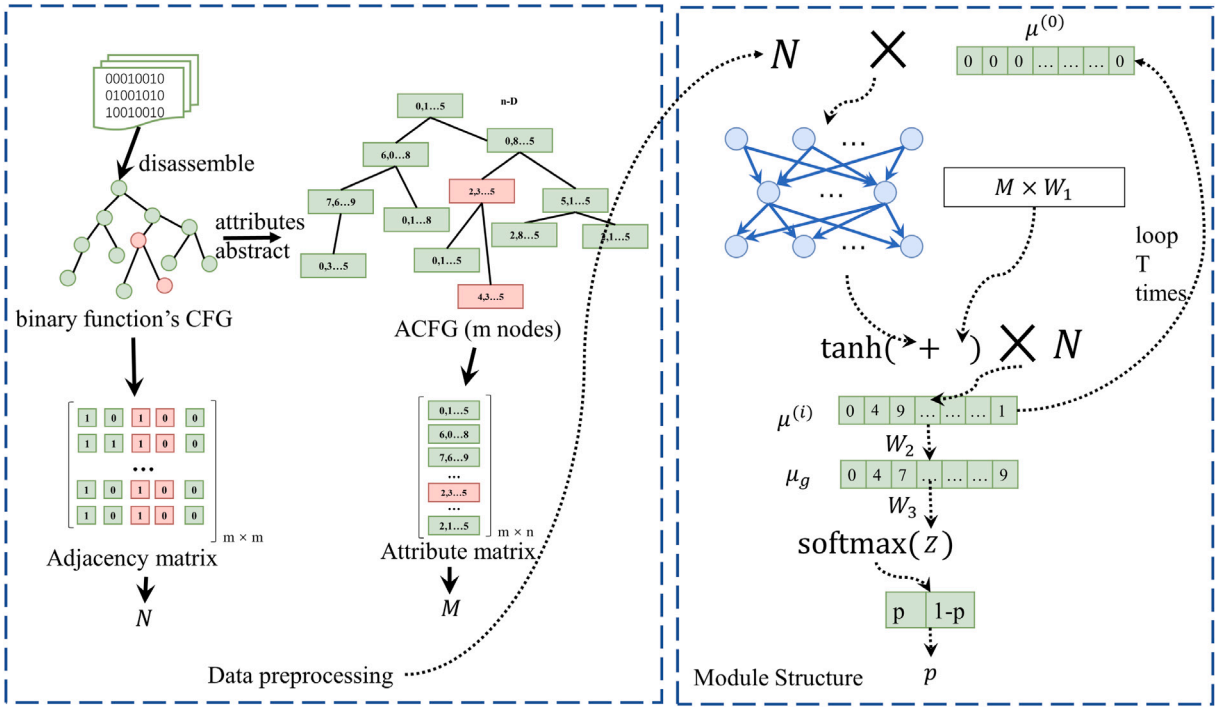


Fig. 4. Details of graph embedding.

Algorithm 1 algorithm of Vulnerable Prediction Model

Input: ACFG g of a function in the binary program
Output: numerical value of the vulnerable prediction p

1: Each g has a adjacency matrix N, and a attribute matrix M. Assume the dimension of ACFG node vector is n and the number of nodes is m, so the dimension of the adjacency matrix is $m \times m$ and the dimension of the attribute matrix is $m \times n$.
2: Initial embedding vector $\mu^{(0)}$ is set to zero. The σ is an n-layer fully connected neural network with parameters $P = P_1, P_2, ...P_n$. Each layer of σ has d neurons.
3: **for** each $i \in T$ **do**
4: $\quad \mu^{(i+1)} \leftarrow tanh(\sigma(N \times \mu^{(i)}) + M \times W_1) \times N$
5: **end for**
6: μ_g means the final graph embedding vector, $\mu_g \leftarrow W_2 \times \mu^{(T)}$
7: Then convert the obtained μ_g into a 2-dimensional vector using the pooling layer, $(p, 1-p) \leftarrow softmax(\mu_g \times W_3)$
8: **return** p

$\mu_g \leftarrow W_3 \times \mu^{(T)}$. The σ is an n-layer fully connected neural network with parameters $P = P_1, P_2, \ldots, P_n$. And we set each layer of σ as d neurons.

To compute the VP of the function, we map the graph embedding vector into a two-dimensional vector $Z = \{z_0, z_1\}$, that is, $Z = W_3 \times \mu_g$, where W_3 is a $2 \times d$ matrix. Then, we use the $softmax$ function to map the value of Z into a vector $Q = \{p, 1-p\}, p \in [0, 1]$, that is, $Q = softmax(Z)$. The value of p is the model's output, representing the vulnerable prediction of the binary program function g. The whole calculating process of the VP is illustrated in Fig. 4. Each function used for training must be assigned a "vulnerable" or "secure" label. Specifically, the ACFG of each function in the binary program is assigned a label of 0 or 1. A label of 0 indicates that the function is secure, while a 1 indicates that the function contains at least one vulnerability. The training process of the vulnerability prediction model results in $VP \in [0, 1]$ serving as the prediction model's output.

4.3. Step3: Basic block value calculation

As discussed previously, training the model labeled data to distinguish and predict the vulnerability probability VP accurately. We assume that a function's vulnerability prediction value (VP) is denoted p_v. let us consider a function f consisting of t basic blocks: $f = b_1, b_2, \ldots, b_t$. For each basic block b_i in f, we assign a basic block value ($BV(b_i)$) calculated by

$$BV(b_i) = k \times p_v + W(b_i) \tag{5}$$

where k is a constant and is a correction factor for the predicted value, the parameter $W(b_i)$ represents the possibility of reaching the basic block b_i based on the branches of the function f, and $W(b_i)$ is calculated to be $\frac{1}{p_b}$. It reflects how difficult it is to reach that particular basic block.

By computing the BV in this manner, VSGFuzz intends to assign higher weights to more vulnerable basic blocks. This approach assists the fuzzier in generating inputs that are more likely to cover these critical basic blocks. We take a function shown in Fig. 5 as an example to describe the logic. In Fig. 5, we observe that the probability of arriving at basic block G is lower than the probability of arriving at basic block F, which has a lower likelihood than basic block H. Thus, we design BV (basic block value) to take both vulnerability probability and the probability of the program execution path into consideration.

4.4. Step4: Seed score

Upon completion of program execution, VSGFuzz records the input execution path. The seed score of the input is calculated as the cumulative sum of the BV values associated with the basic blocks along the execution path. Suppose the execution path is defined as $path : b_1 \rightarrow b_3 \rightarrow b_6 \rightarrow b_8$. Consequently, the corresponding reward value for this input can be expressed as

$$SeedScore = BV(b_1) + BV(b_3) + BV(b_6) + BV(b_8) + C \times HugeScore \tag{6}$$

where C is a binary parameter to control the seed score, if a crash occurred, C is set to 1; otherwise, C is set to 0. $HugeScore$ is set to be a large numerical value that significantly contributes to the reward value in a crash.

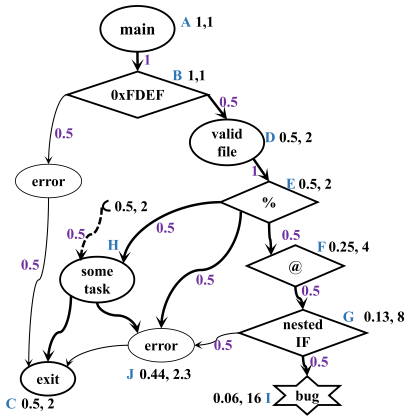


Fig. 5. The Vuzzer block value calculation.

Table 1
Mutate actions and descriptions.

Mutate actions	Description
Eliminate-random	Random elimination
Change-bytes	Modify a single byte
Add-random	Random how to add content
Change-random	Random modification
Single-change-random	Randomly modify a single byte
Lower-single-random	Convert random single byte content to lowercase
Raise-single-random	Boost single random
Eliminate-null	Randomly eliminate the content is null
Eliminate-double-null	Perform random small elimination twice and the content is null
Totally-random	Completely free variation
Int-slide	Mutate int type
Double-fuzz	Combination of any two fuzzing techniques
Change-random-full	Random modification of the whole range
Crossover	Cross mutation of two files

4.5. Step5: Dynamic execution

VSGFuzz is a repetitive process. It maintains a seed pool to store high-quality inputs as seeds. Initially, VSGFuzz executes the binary program using an initial input provided by the user. Concurrently, it utilizes Dynamic Binary Instrumentation (DBI) to track program execution information, including basic block coverage. Leveraging BV and execution information, VSGFuzz calculates a seed score for each executed test case (i.e., input). Test cases with higher seed scores are considered high-quality inputs and added to the seed bank. Test cases that trigger crashes are also included in the seed pool and assigned higher seed scores. Next, VSGFuzz generates next-generation test cases by mutating the seeds present in the seed pool. The new seeds are also stored in the seed pool. Then, we repeat the process to select a seed from the pool to run a fuzzing test.

In general, control flow is used to specify path priority and identify error-handling blocks, and data flow is utilized to determine the location of variation and specific operations. Specifically, AFL is improved to give greater weight to paths of interest. Based on analyzing the basic block weight calculation in line with Markov properties, we conclude that the transfer probabilities between blo-cks are independent, and their probabilities are only related to connected blocks, so the inverse of the probability is set as the weight. In the paper, error-handling blocks are designed with weights set to negative numbers, and the sum of the weights of the blocks on the path is defined as the path weights. The higher the score, the higher the weight and the higher priority the input corresponding to the path is considered to have, and it will be preferred for seed mutation and being selected again as the seed file. Vuzzer recognizes magic bytes by Pin insertion, and through continuous iteration, it matches the information in *cmp* and address fetching class operation *lea* with the seed file input and the newly generated inputs and finds the magic bytes in the input to find the possible locations of magic bytes and mutate them more accurately.

4.6. Step6: RL for fuzzing

4.6.1. Modeling based on RL

This section introduces a methodology for constructing the state, action, reward and environment components in reinforcement learning (RL) problems, which takes inspiration from the operations and environment of fuzzing. The Markov property is satisfied by ensuring that the system's state in the subsequent moment is solely dependent on its current state and is not influenced by other states. Within the context of fuzzing, the mutation strategy is utilized to select an appropriate mutating operation for the current seed. Subsequently, the target program processes the mutant seed, and the specific reward is computed based on the runtime feedback received.

The RL process comprises four fundamental elements: state, action, reward, and environment. To effectively utilize the RL algorithm in optimizing the fuzzing process, it is crucial to abstractly model the fuzzing process as a problem solvable by RL. The subsequent discussion will focus on introducing the abstraction and selection process of the state, action, and reward elements in the fuzzing problem modeling based on RL.

State:. In the context of fuzzing, the effectiveness of testing primarily relies on the generation of high-quality seeds, where high-quality seeds refer to those that uncover more vulnerabilities in the program's state. In this approach, the input seeds serve as the state. A byte array method is adopted to represent these input seeds, where the state s corresponds to the input seed data D. Each element of s has a value range of $[0, 255]$, and the maximum length of the array is denoted $Lmax$, which depends on the specific problem environment. If the length Lp of the seed data D is less than $Lmax$, it is padded with 0 to reach the maximum size.

$$D(x) = \begin{cases} d_i, & i \in [0, L_D] \\ 0, & i \in (L_D, L_{max}] \end{cases} \tag{7}$$

Action:. The fundamental objective of the fuzzing process is to mutate [40] the seed data, thereby generating new seeds capable of inducing abnormal states in the target program. This research devises a variation action space denoted as A, consisting of 14 distinct actions. The detailed list of these actions is presented in Table 1.

The RL model employs a strategy π to select a variation action a_t from the variation space A, based on the current state s_t, which can be mathematically represented as

$$a_t = \pi(s_t) \tag{8}$$

Subsequently, the system applies the selected action to mutate the current input data state s_t, facilitating a comprehensive exploration of both the environment state space and the mutation action space. As a result, the system acquires the corresponding state s'_t of the mutated seed, which exhibits an enhanced vulnerability triggering effect, formally expressed as

$$s'_t = Mutate(s_t, a_t) \tag{9}$$

Reward:. In the conventional fuzzing process, the evaluation of test success typically revolves around whether the program's potential vulnerabilities are triggered, monitoring the occurrence of abnormal states such as crashes or hangs in the target program being tested. However, the process of provoking program exceptions through fuzzing can be time-consuming. Suppose this information is solely used as feedback for the environment. In that case, the fuzzing cannot promptly adjust

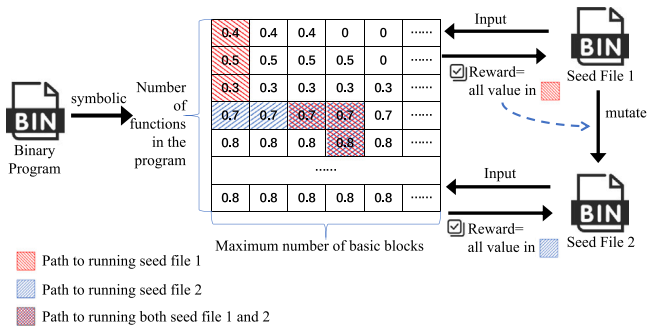


Fig. 6. Vulnerable state matrix for the binary program.

the mutation strategy, resulting in wasteful resource utilization on meaningless seed mutations.

In our approach, the reward for reinforcement learning training is determined by the Seed Score of the fuzzier, which is defined in the seed selection strategy section of 4.3.

In this paper, we use the following Fig. 6 to understand how our design's basic block score values are calculated and how the design guides the seeds to mutate towards obtaining higher values. We assign a larger number to the vulnerability basic block to guide the fuzzing towards fragile components. At first, we understand binary program analysis as a basic block value matrix. Each row represents a function within the binary program, while the length of the columns represents the maximum number of basic blocks found in functions. Thus, each cell in the state matrix signifies the basic block's final score. Consequently, we can component the vulnerability probability of the binary program to a matrix and employ this matrix to help calculate the Seed Score, which guides the mutation operation of the fuzzier.

Environment:. As shown in Fig. 2, we use the runtime coverage feedback information as a part of the reinforcement learning environment component. We can simulate the fuzzing test running environment by combining the feedback and state information.

4.6.2. Strategy selection of reinforcement learning algorithm

To effectively select mutation operations for fuzzing, choosing RL algorithms that can handle large state and action spaces is essential. In 2014, David Silver et al. proposed the deterministic policy gradient (DPG) algorithm, which demonstrated the existence of a deterministic policy gradient through mathematical analysis. Subsequently, the deep deterministic policy gradient (DDPG) algorithm [41] emerged by combining the DPG algorithm with deep neural networks, enhancing its practicality. Given the relatively large mutation operation space and input seed state space in the fuzzing process model, we opt for the DDPG algorithm to construct the RL model [42].

The DDPG algorithm comprises four neural networks, which are employed to approximate the Q-value function and policy. The Critic target network is responsible for approximating the state–action Q-value function, denoted as $Q_{\omega'}(S_{t+1}, \pi_{\theta'}(S_{i+1}))$, at the next time step. Here, the next action value is passed through the Actor target network to obtain the approximate estimation $\pi_{\theta'}(S_{t+1})$. Subsequently, the target value for the Q-value function in the current state can be obtained as follows:

$$y_i = r_i + \gamma Q_{\omega'}(S_{t+1}, \pi_{\theta'}(S_{i+1})) \tag{10}$$

The critic training network is responsible for outputting the Q-value function $Q_\omega(S_t, a_t)$ for the current state–action pair, which is utilized to evaluate the current strategy. DDPG incorporates a Gaussian noise function into the behavior policy to enhance the agent's exploration within the environment. The objective of the Critic network is defined as the difference between the target value and the predicted Q-value, denoted as

$$y_i - Q_\omega(S_i, a_i) \tag{11}$$

The parameters of the Critic network are updated by minimizing the loss value, which is computed using the mean square error loss function. The loss function for the update of the Critic network is defined as follows:

$$loss = \frac{1}{N}\sum_i (y_i - Q_\omega(S_i, a_i))^2 \tag{12}$$

Here, $a_i = \pi_\theta(S_i) + \varepsilon$, where ε represents the exploration noise added to the behavior policy.

The Actor target network provides the strategy for the next state, while the Actor training network provides the plan for the current state. By leveraging the Q-value function from the Critic training network, we can obtain the gradient of the Actor's strategy when updating its parameters:

$$\nabla_{\pi_\theta} J = \frac{1}{N}\sum_i \nabla_a Q_\omega(s, a)|s = s_i, a = \pi\theta(s_i)\nabla_\theta \pi_\theta(s)|_{s_i} \tag{13}$$

To update the parameters of the target networks ω' and θ', DDPG employs a soft update mechanism, ensuring that the parameters are gradually updated by only updating a portion of the parameters at each learning step. This soft update mechanism improves the stability of the learning process:

$$\begin{aligned} \omega' &\leftarrow \xi\omega + (1-\xi)\omega' \\ \theta' &\leftarrow \xi\theta + (1-\xi)\theta' \end{aligned} \tag{14}$$

DDPG has value function-based and policy-based method features, which enable deep reinforcement learning to handle continuous actions and have certain exploratory capabilities. The pseudocode 2 is the DDPG algorithm:

Algorithm 2 DDPG algorithm

1: Randomly initialize critic network $Q(a, a|\theta^Q)$ and actor $\mu(s|\theta^\mu)$ with weights θ^Q and θ^μ.
2: Initialize target network Q' and μ' with weights $\theta^{Q'} \leftarrow \theta^Q$, $\theta^{\mu'} \leftarrow \theta^\mu$
3: Initialize replay buffer R
4: **for** eposide = 1, M **do**
5: Initialize a random process N for action exploration
6: Receive initial observation state s_1
7: **for** $t = 1, T$ **do**
8: Select action $a_t = \mu(s_t|\theta^\mu) + N_t$ according to the current policy and exploration noise
9: Execute action a_t and observe reward r_t and observe new state s_{t+1}
10: Store transition (s_t, a_t, r_t, s_{t+1}) in R
11: Sample a random minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from R
12: Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^Q)$
13: Update critic by minimizing the loss: $L = \frac{1}{N}\sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$
14: Update the actor policy using the sampled policy gradient: $\nabla_{\theta^\mu} J \approx \frac{1}{N}\sum_i \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu}\mu(s|\theta^\mu)|s_i$
15: Update the target networks:
$$\theta^{Q'} \leftarrow \tau\theta^Q + (1-\tau)\theta^{Q'}$$
$$\theta^{\mu'} \leftarrow \tau\theta^\mu + (1-\tau)\theta^{\mu'}$$
16: **end for**
17: **end for**

Table 2
Parameter settings for the vulnerability prediction model.

Name	Value
Max-nodes	61
Epochs	5
Max-nodes-threshold	0
Step-per-epoch	5000
Buffer-size	1000
Valid-step-per-epoch	3000
Mini-batch	10
Test-per-epoch	3000
Learning-rate	0.0001
Embedding-size	256
Embedding-depth	2

Table 3
Environment setting.

System hardware environment	Specific configuration
CPU	Intel(R) Core(TM) i5-10500 CPU, 3.10 GHz
Memory	RAM (16.0 GB)
Hardware	1T
GPU	0

5. Implementation and settings

5.1. Graph embedding-based vulnerability prediction model

As shown in the previous design, we need to accomplish two parts to implement a graph embedding-based vulnerability prediction model. Those two parts are (1) an ACFG extractor and (2) a vulnerability prediction model. The ACFG extractor was implemented by a plugin provided by the widely used disassembly tool IDA Pro [43]. The vulnerability prediction model was developed using the popular deep learning framework TensorFlow.

Based on the Genius project [35], the ACFG extractor for binary program functions was developed. By modifying and enhancing the feature extraction method of Genius, we can customize and incorporate the specific feature data required for our purposes. The vulnerability prediction model takes the ACFG of the function along with its corresponding label as input. Therefore, during model training, we require a dataset consisting of functions with known vulnerabilities. This experiment uses the Juliet Test Suite as the training dataset, widely used in various vulnerability-related studies. The Juliet Test Suite, published by the National Institute of Standards and Technology (NIST), comprises C/C++ language programs, where each function is labeled as "good" (indicating "secure") or "bad" (indicating "vulnerable").

Our parameter settings are shown in Table 2 based on the training data analysis and the model's tuning.

In terms of the system platform, we utilized the hardware environment detailed in Table 3.

5.2. RL based and vulnerable states guided fuzzing

In the paper, we developed a fuzzier VSGFuzz based on VUzzer [16], an advanced evolutionary fuzzer designed for binary targets. Each fuzzing experiment was performed on a virtual machine running Ubuntu 20.04 LTS. The virtual machine had a 64-bit single-core Intel CPU running at 4.2 GHz and 4 GB of RAM.

RL model training and fuzzing efficiency are closely related to its hardware and software environment. For the training model, we set the hardware environment in Table 3, and the software environment is shown in Table 4. We also use library files such as Tensorflow, Keras-RL2, and Gym to build a RL model based on the DDPG algorithm.

6. Evaluation

In this section, we evaluate VSGFuzz through the following two aspects: (1) the evaluation results of the vulnerability prediction model and (2) the evaluation results of the RL-based fuzzer.

6.1. Graph embedding based vulnerability prediction model evaluation

Evaluation metrics:. Accuracy, loss, ACFG extraction time, and training time were used to measure the performance of the vulnerability prediction model. Table 5 summarizes our evaluation results. We use the cross entropy loss function to calculate the loss of the model. The VPModel-evaluation table records the loss rate and accuracy of the model. We use the cross-entropy loss function to calculate the loss of the model, which quickly drops to a low value, then stabilizes, and the surface model rapidly converges. The accuracy curve of the model is more than 0.995, which means that the model can correctly distinguish between positive and negative cases to a large extent.

The figure (Fig. 7) depicts the reward graph generated during the training of a fuzzy test model using the Deep Q Network (DQN) and DDPG. The orange line represents the reward values obtained by DDPG, while the brown line represents the reward values obtained by DQN. The graph illustrates that DDPG outperforms the DQN algorithm regarding model training performance. The higher reward values achieved by DDPG indicate its superior performance in optimizing fuzzy test models. The red dashed line at the top of the chart represents the maximum reward value attained under the DDPG model, while the blue dashed line represents the maximum reward value achieved under the DQN model. The reward values are designed to be the seed score after the seed is executed.

The abscissa in the figure (Fig. 7) represents the number of execution steps during the fuzz testing process, and the ordinate represents the reward feedback value obtained. The image analysis shows that in the early stage of fuzz testing, a higher reward value feedback is not obtained each time a new seed file is executed. However, about 700 steps after triggering, the feedback value suddenly increased significantly. It is speculated that it may be because the seed file generated by the mutation triggered a certain critical path at this stage, which may have been marked as possible in previous predictions. Loopholes. Subsequently, it remains stable for some time, possibly because the mutations performed during this period were local-scale changes that failed to induce substantial or larger-scale path changes. It should be emphasized that the changes in the two lines shown in the figure all come from changes in path coverage, and there is no program crash event. In the later experiments, we also used DDPG to train the model.

DDPG consists of four neural networks for approximate representation of Q-valued functions and strategies. The Critic target network is used to approximate the Q-value function of the state action at the next moment: $Q_{\omega'}\left(S_{t+1}, \pi_{\theta'}\left(S_{i+1}\right)\right)$, The Actor target network approximates the next action value $\pi_{\theta'}\left(S_{t+1}\right)$ The target value of the Q-value function in the current state can then be obtained:

$$y_i = r_i + \gamma Q_{\omega'}\left(S_{t+1}, \pi_{\theta'}\left(S_{i+1}\right)\right)$$

The Critic training network outputs the Q value function of the current moment state–action $Q_{\omega}\left(s_t, a_t\right)$, which is used to evaluate the current strategy. To increase the agent's exploration in the environment, DDPG adds a Gaussian noise function to the behavior strategy. The goals of the Critic network are defined as $y_i - Q_{\omega}\left(s_i, a_i\right)$.

Update the parameters of the Critic network by minimizing the loss value (mean square error loss); the loss function when the Critic network is updated is:

$$loss = \frac{1}{N}\sum_{i}\left(y_i - Q_{\omega}\left(s_i, a_i\right)\right)^2$$

where $a_i = \pi_{\theta}\left(s_i\right) + \varepsilon, \varepsilon$ represents the exploration noise on the behavioral strategy.

Table 4
Package version setting of software environment.

software	Ubuntu	Python	Gym	Tensorflow	Keras-RL2	Matplotlib	numpy	pip
Version	20.04	3.8.5	0.26.1	2.10.0	1.0.5	3.7.1	1.24.2	23.1

Table 5
Evaluation results of the vulnerable prediction model.

Evaluation index	Results
Accuracy	99.8%
Loss	0.011
Model training time	About 200 min to train an effective model
ACFG extraction time	average 1.8 s to extract ACFG of a binary program

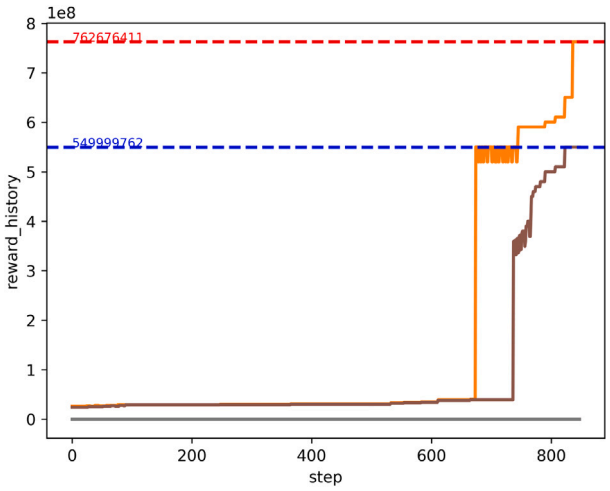


Fig. 7. Reward for DDPG and DQN. (The Orange line represents DDPG and the brown line represents DQN.). (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

The target network of the Actor is used to provide the policy of the next state, while the training network of the Actor is used to give the policy of the current state. Combining the Q value function of the training network of Critic, the policy gradient of Actor when parameters are updated can be obtained:

$$\nabla_{\pi_\theta} J = \frac{1}{N} \sum_i \nabla_a Q_\omega(s,a)\Big|_{s=s_i a=\pi_\theta(s_i)} \nabla_\theta \pi_\theta(s)\Big|_{s_i}$$

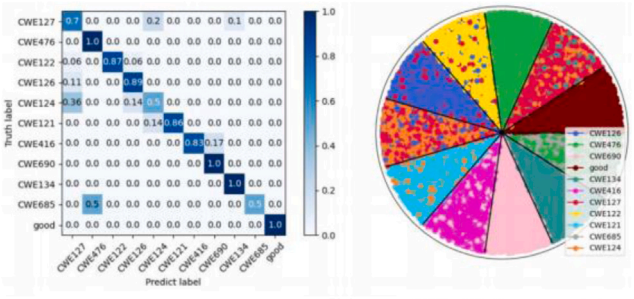
For the update of the target network parameters ω' and θ', DDPG ensures that the parameters can be updated slowly through a soft update mechanism (update some parameters every time you learn), thus improving the stability of learning:

$$\omega' \leftarrow \xi\omega + (1-\xi)\omega'$$
$$\theta' \leftarrow \xi\theta + (1-\xi)\theta'$$

In the pre-training stage of fuzzy test, the basic idea of this experiment is to classify the input data to determine the type of the input. The vulnerability identification model is then evaluated using the confusion matrix. The correct classification samples (true labels = prediction labels) are distributed diagonally across the confusion matrix. It can be seen from Figure (Fig. 8)(a) that this vulnerability identification model has a good classification recognition ability, and its diagonal values are above 50% of the classification success rate. Figure (Fig. 8) shows the confusion matrix table of the training model and the classification scatter plot of the prediction results. Figure (Fig. 8)(b) presents a classification scatter plot of the predicted results to more clearly show the correctly classified results. It is observed from the figure that for CWE476, CWE690, CWE134, good and other categories, the model achieves 100 percent classification accuracy. Specifically,

Table 6
The number of crashes found for 24 h in LAVA-M datasets.

programs	VSGFuzz(64 bit)	Vuzzer	V-FUZZ	AFL	Suzzer
base64	**28**	17	27	0	18
uniq	**28**	27	28	0	5
who	**67**	50	66	0	40



(a)Classification confusion matrix (b)Classification scatter plot

Fig. 8. Classification results evaluation chart. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

these categories do not contain other colors within the color region in the diagram, indicating that the model is highly accurate in identifying these specific categories.

6.2. RL based and vulnerable states guided fuzzing evaluation

In this section, we demonstrate VSGFuzz's ability of discovering vulnerabilities on three different datasets: A. LAVA-M Dataset, B. DARPA CGC binaries and C. Real-Word Dataset. The results demonstrated that VSGFuzz was able to not only precisely evaluate vulnerability probability to detect more vulnerabilities, but also consume less computation time with a promising efficiency.

A. LAVA-M Dataset

Table 6 shows the number of bugs found by VSGFuzz, Vuzzer, V-Fuzz, AFL, and Suzzer, respectively. The program vulnerability numbers triggered by the LAVA-M program during the fuzz testing process are as shown in the following Table 7. Furthermore, VSGFuzz can trigger several unlisted bugs and exhibits better performance than V-Fuzz in this case. VSGFuzz excels in terms of coverage as shown in Table 8. Compared to other methods, it can explore a wider range of execution paths within the program.

B. DARPA CGC Dataset

We obtained a total of 149 CGC binaries. However, we were unable to run VSGFuzz on all of these devices due to the following reasons:

(1) All binaries are interactive by accepting input from STDIN. Once launched, many display a menu to select actions, including an exit option. Additionally, there are many menus (in different states of the program) with varying options for exit. Since VSGFuzz requires one step to complete with completely random input, performing such input would put the application in a loop looking for valid possibilities, including an option to exit, which would cause the application to run forever. This is an interface issue, not a fundamental limitation of our fuzzing approach.

(2) Some binaries involve interactions with others, which CSGFuzz cannot handle.

(3) Since VSGFuzz is based on Pin, we cannot run some binaries with Pin.

Table 7
Vulnerability number triggered in LAVA-M datasets.

Programs	Triggered bug number
base64	255,562,560,572,582,790, 776,788,782,386,235,222, 576,276,780,778,786,558, 583,774,1,566,253,584, 784,554,573,556
uniq	321,170,293,471,346,171, 443,112,368,347,322,318 297,393,169,372,446,396, 447,296,371,215,166,397, 472,222,130,468
who	1,3,4,5,7,8, 9,10,14,18,22,26, 56,58,60,62,75,79, 83,87,109,110,111,112, 113,114,115,116,118,119, 120,121,122,123,124,126, 127,137,138,159,319,1105, 1229,1691,1692,1969,2888,3947, 4145,4358

Table 8
Comparing edge coverages of fuzzers for 24 h runs in LAVA-M datasets.

programs	VSGFuzz	Vuzzer	AFL-Fast	AFL
readelf -a	**8890**	12	1073	746
nm -C	**2562**	221	1503	1418
objdump -D	**2410**	307	263	257
size	**2361**	541	1924	1236
strip	**4012**	478	960	856
libjpeg	**1654**	60	651	94
libxml	**1951**	16	392	517
mupdf	**443**	38	371	370
zlib	**502**	15	371	374
harfbuzz	**7021**	111	4021	3255

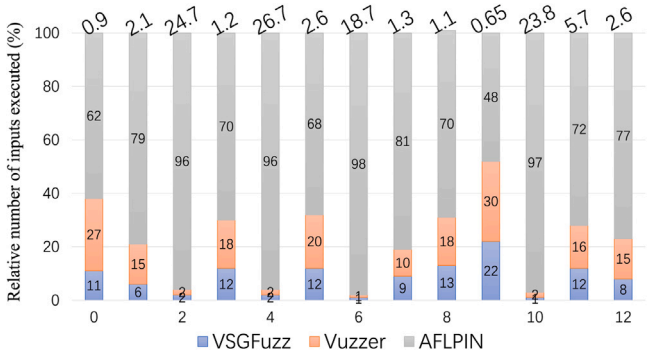


Fig. 9. Relative number of inputs executed for each of the CGC Binaries, wherein both VSGFuzz and VUzzer and AFLPIN find crashes. The numbers above the bars are the total number of inputs (in thousands) executed.

After accounting for the obstacles mentioned above, we ran a total of 63 binaries, and for comparison, we also ran AFLPIN and VUzzer. VSGFuzz found crashes in 37 CGC binaries, AFLPIN found only 23, and VUzzer found crashes in 29 CGC binaries. Since each CGC also ships with a patched version, we verified the bugs found by VSGFuzz by running the patched version of the binary to observe whether there were further crashes. The most important result is the number of inputs executed per crash among the three fuzzers. We used it for up to 6 h, and the graph (Fig. 9) depicts the number of executions in the case where all three fuzzes found crashes (13 times in total), which shows that VSGFuzz can significantly reduce the search space compared to the other two methods.

Table 9
The number of crashes found for 24 h in Real-Word datasets.

Programs	VSGFuzz(64 bit)	Vuzzer	AFL	Suzzer
mpg321	**425**	337	19	237
gif2png	**189**	127	7	40
pdf2svg	**24**	13	0	2
tcpdump	**12**	3	0	57
tcptrace	**459**	403	238	16
djpeg	**1**	1	0	1

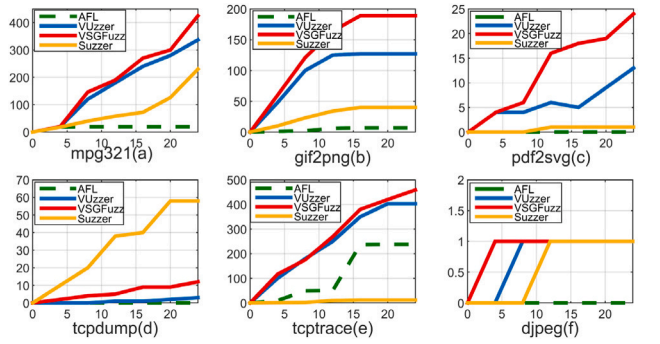


Fig. 10. 24-h crash number statistics chart. (The solid red line represents VSGFuzz, the solid blue line represents VUzzer, the dotted green line represents AFL, and the solid yellow line represents Suzzer). (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

C. Real-Word Dataset

We evaluate the performance of VSGFuzz using a real-world dataset (djepg/eog, tcpdump, tcptrace, pdf2svg, mpg321, gif2png) and compare with Vuzzer, AFL, and Suzzer, shown in Table 9. The fuzzing time for each program was up to 24 h. To highlight the performance of VSGFuzz, we also ran AFL, VUzzer and Suzzer on these programs(Suzzer [34] predicts the vulnerable probability of functions based on a deep learning-based model and gives each basic block in the vulnerable function a static score. Then, for each input, the sum of the static score of all the basic blocks on its execution path is calculated, and the inputs are prioritized with higher scores). The table below shows the results of running on real programs regarding the number of unique crashes found. VSGFuzz significantly outperforms the other two methods.

The figure depicts (Fig. 10) the crashes of 6 programs within 24 h. The x-axis of each graph shows the cumulative sum of crashes sampled every 2 h. For almost every application, VSGFuzz can find crashes in continuous iterations of fuzz testing. Compared with the other three methods, VSGFuzz can find crashes and detect crashes in less time.

7. Conclusion

This paper presents an innovative fuzzing method named VSGFuzz, designed for vulnerability analysis in binary programs. Our approach comprises several key steps. Firstly, we develop a vulnerability prediction model to estimate the vulnerability probability of individual functions within a binary program. Next, we leverage these vulnerability probabilities and basic block values to compute seed file scores. Subsequently, a reinforcement learning model is trained to guide the mutation operations applied to the seed file, enhancing both the quality and efficiency of the fuzz testing process. Finally, our test results on the LAVA-M dataset, DARPA CGC dataset and Real-Word Dataset highlight the effectiveness of VSGFuzz. Moreover, the training results on real programs demonstrate the tool's usability, especially in enhancing program coverage. These findings underscore the effectiveness of our approach to identifying vulnerabilities and fortifying the security of binary programs.

CRediT authorship contribution statement

Guoyan Cao: Writing – original draft, Writing – review & editing, Methodology, Conceptualization. **Yanhui Ma:** Writing – original draft, Software, Methodology. **Mengjiao Geng:** Software, Data curation.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

Funded by Open Foundation of Key Laboratory of Cyberspace Security, Ministry of Education of China (No. KLCS20240202).

Data availability

No data was used for the research described in the article.

References

[1] Cameron A, Alam A, Anjum N, Khan JA, Mylonas A. STATOS: A portable tool for secure malware analysis and sample acquisition in low resource environments. Array 2025;26(100391):1–15, URL https://www.sciencedirect.com/science/article/pii/S2590005625000189?via%3Dihub.

[2] Böhme M, Pham V-T, Nguyen M-D, Roychoudhury A. Directed greybox fuzzing. In: Proceedings of the 2017 ACM SIGSAC conference on computer and communications security. 2017, p. 2329–44.

[3] Cimpanu C. Hacker leaks passwords for more than 500,000 servers, routers, and iot devices. 2020.

[4] Shin Y, Williams L. Can traditional fault prediction models be used for vulnerability prediction? Empir Softw Eng 2013;18:25–59.

[5] Salahirad A, Gay G, Mohammadi E. Mapping the structure and evolution of software testing research over the past three decades. J Syst Softw 2023;195:111518. http://dx.doi.org/10.1016/j.jss.2022.111518, URL https://www.sciencedirect.com/science/article/pii/S0164121222001947.

[6] Miller BP, Fredriksen L, So B. An empirical study of the reliability of UNIX utilities. Commun ACM 1990;33(12):32–44.

[7] Forrester JE, Miller BP. An empirical study of the robustness of windows NT applications using random testing. In: Proceedings of the 4th conference on USeNIX windows systems symposium - volume 4. WSS '00, USA: USENIX Association; 2000, p. 6.

[8] Takanen A, Demott JD, Miller C, Kettunen A. Fuzzing for software security testing and quality assurance. Artech House; 2018.

[9] Godefroid P, Levin MY, Molnar DA, et al. Automated whitebox fuzz testing. In: NDSS. Vol. 8, 2008, p. 151–66.

[10] Wang T, Wei T, Gu G, Zou W. TaintScope: A checksum-aware directed fuzzing tool for automatic software vulnerability detection. In: 2010 IEEE symposium on security and privacy. IEEE; 2010, p. 497–512.

[11] Zhang D, Liu D, Lei Y, Kung D, Csallner C, Nystrom N, Wang W. SimFuzz: Test case similarity directed deep fuzzing. J Syst Softw 2012;85(1):102–11. http://dx.doi.org/10.1016/j.jss.2011.07.028, Dynamic Analysis and Testing of Embedded Software. URL https://www.sciencedirect.com/science/article/pii/S016412121100197X.

[12] American fuzzy lop (AFL). 2013, http://lcamtuf.coredump.cx/afl/, [Accessed 20 October 2023].

[13] Haller I, Slowinska A, Neugschwandtner M, Bos H. Dowsing for {Overflows}: A guided fuzzer to find buffer boundary violations. In: 22nd USENIX security symposium. USENIX security 13, 2013, p. 49–64.

[14] Stephens N, Grosen J, Salls C, Dutcher A, Wang R, Corbetta J, Shoshitaishvili Y, Kruegel C, Vigna G. Driller: Augmenting fuzzing through selective symbolic execution. In: NDSS. Vol. 16, 2016, p. 1–16.

[15] Grieco G, Ceresa M, Mista A, Buiras P. QuickFuzz testing for fun and profit. J Syst Softw 2017;134:340–54. http://dx.doi.org/10.1016/j.jss.2017.09.018, URL https://www.sciencedirect.com/science/article/pii/S0164121217302066.

[16] Rawat S, Jain V, Kumar A, Cojocar L, Giuffrida C, Bos H. Vuzzer: Application-aware evolutionary fuzzing. In: NDSS. Vol. 17, 2017, p. 1–14.

[17] Lee S, Han H, Cha SK, Son S. Montage: A neural network language model-guided javascript engine fuzzer. In: Proceedings of the 29th USENIX conference on security symposium. 2020, p. 2613–30.

[18] Li Y, Ji S, Lyu C, Chen Y, Chen J, Gu Q, Wu C, Beyah R. V-fuzz: Vulnerability prediction-assisted evolutionary fuzzing for binary programs. IEEE Trans Cybern 2020;52(5):3745–56.

[19] Abaimov S, Bianchi G. A survey on the application of deep learning for code injection detection. Array 2021;11(100077):1–18, URL https://www.sciencedirect.com/science/article/pii/S2590005621000254?via%3Dihub.

[20] Lv C, Ji S, Li Y, Zhou J, Chen J, Zhou P, Chen J. SmartSeed: Smart seed generation for efficient fuzzing. 2018, ArXiv abs/1807.02606. URL https://api.semanticscholar.org/CorpusID:49656556.

[21] Wu Z, Johnson E, Yang W, Bastani O, Song D, Peng J, Xie T. REINAM: reinforcement learning for input-grammar inference. In: Proceedings of the 2019 27th acm joint meeting on European software engineering conference and symposium on the foundations of software engineering. 2019, p. 488–98.

[22] Luong NC, Hoang DT, Gong S, Niyato D, Wang P, Liang Y-C, Kim DI. Applications of deep reinforcement learning in communications and networking: A survey. IEEE Commun Surv Tutor 2019;21(4):3133–74.

[23] Deng Y, Xia CS, Peng H, Yang C, Zhang L. Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models. In: Proceedings of the 32nd ACM SIGSOFT international symposium on software testing and analysis. 2023, p. 423–35.

[24] Cheng L, Zhang Y, Zhang Y, Wu C, Li Z, Fu Y, Li H. Optimizing seed inputs in fuzzing with machine learning. In: 2019 IEEE/ACM 41st international conference on software engineering: companion proceedings. ICSE-companion, IEEE; 2019, p. 244–5.

[25] She D, Pei K, Epstein D, Yang J, Ray B, Jana S. Neuzz: Efficient fuzzing with neural program smoothing. In: 2019 IEEE symposium on security and privacy. SP, IEEE; 2019, p. 803–17.

[26] Scott J, Sudula T, Rehman H, Mora F, Ganesh V. BanditFuzz: fuzzing SMT solvers with multi-agent reinforcement learning. In: International symposium on formal methods. Springer; 2021, p. 103–21.

[27] Böttinger K, Godefroid P, Singh R. Deep reinforcement fuzzing. In: 2018 IEEE security and privacy workshops. SPW, IEEE; 2018, p. 116–22.

[28] Liu X, Prajapati R, Li X, Wu D. Reinforcement compiler fuzzing. 2019, ArXiv. URL https://api.semanticscholar.org/CorpusID:156053247.

[29] Drozd W, Wagner MD. FuzzerGym: A competitive framework for fuzzing and learning. 2018, ArXiv abs/1807.07490. URL https://api.semanticscholar.org/CorpusID:49876120.

[30] Serebryany K. libFuzzer a library for coverage-guided fuzz testing. 2023, https://llvm.org/docs/LibFuzzer.html.

[31] Kuznetsov A, Yeromin Y, Shapoval O, Chernov K, Popova M, Serdukov K. Automated software vulnerability testing using deep learning methods. In: 2019 IEEE 2nd Ukraine conference on electrical and computer engineering. UKRCON, IEEE; 2019, p. 837–41.

[32] Zhang G, Wang P, Yue T, Kong X, Huang S, Zhou X, Lu K. Mobfuzz: Adaptive multi-objective optimization in gray-box fuzzing. In: Network and distributed systems security (NDSS) symposium. Vol. 2022, 2022.

[33] Wang Y, Wu Z, Wei Q, Wang Q. Neufuzz: Efficient fuzzing with deep neural network. IEEE Access 2019;7:36340–52.

[34] Zhao Y, Li Y, Yang T, Xie H. Suzzer: A vulnerability-guided fuzzer based on deep learning. In: International conference on information security and cryptology. Springer; 2020, p. 134–53.

[35] Cai H, Zheng VW, Chang KC-C. A comprehensive survey of graph embedding: Problems, techniques, and applications. IEEE Trans Knowl Data Eng 2018;30(9):1616–37.

[36] Li Z, Zou D, Xu S, Ou X, Jin H, Wang S, Deng Z, Zhong Y. Vuldeepecker: A deep learning-based system for vulnerability detection. 2018, arXiv preprint arXiv:1801.01681.

[37] Feng Q, Zhou R, Xu C, Cheng Y, Testa B, Yin H. Scalable graph-based bug search for firmware images. In: Proceedings of the 2016 ACM SIGSAC conference on computer and communications security. 2016, p. 480–91.

[38] Xu X, Liu C, Feng Q, Yin H, Song L, Song D. Neural network-based graph embedding for cross-platform binary code similarity detection. In: Proceedings of the 2017 ACM SIGSAC conference on computer and communications security. 2017, p. 363–76.

[39] Shen X, Chung F-L. Deep network embedding for graph representation learning in signed networks. IEEE Trans Cybern 2020;50(4):1556–68.

[40] Lyu C, Ji S, Zhang C, Li Y, Lee W-H, Song Y, Beyah R. MOPT: Optimized mutation scheduling for fuzzers. In: USENIX security symposium. 2019, p. 1949–66.

[41] Lillicrap TP, Hunt JJ, Pritzel A, Heess N, Erez T, Tassa Y, Silver D, Wierstra D. Continuous control with deep reinforcement learning. 2015, arXiv preprint arXiv:1509.02971.

[42] Zhang Z, Cui B, Chen C. Reinforcement learning-based fuzzing technology. In: Innovative mobile and internet services in ubiquitous computing: proceedings of the 14th international conference on innovative mobile and internet services in ubiquitous computing. IMIS-2020, Springer; 2021, p. 244–53.

[43] Eagle C. The IDA pro book. no starch Press; 2011.